

SHIFT11

Phase-Wise Development Plan

Revised roadmap aligned with the current README and synthetic-data corrections.

Working plan, not a claim that every listed component is already implemented.

Core principle: deterministic analytics establish what happened; AI explains why it mattered.

Product promise: detect the shift, attribute the players, reconstruct the evidence, and explain why the match changed.

Delivery rules

- Build in dependency order. Do not implement momentum scoring on top of a generator that produces incoherent possessions or incompatible coordinates.
- Keep analysis deterministic and inspectable. The AI layer receives verified facts and must not invent events, metrics, or player contributions.
- Treat all momentum weights and shift thresholds as starting hypotheses. Calibrate them with controlled scenarios and tests; do not describe them as established football-science standards.
- Use seeded data for reproducibility. A scenario label configures generation but does not force the analytics engine to report a shift.

Roadmap

PHASE 1 | PROJECT FOUNDATION

Goal: Make the local project and its data contract repeatable.

- Python project structure, virtual environment, Git/GitHub workflow and clear module boundaries.
- Team, player, formation, match and event models; JSON serialization and a known output location.
- Confirm how to run the generator and tests from the project root.

Exit check: The same seed and configuration produce the same ordered event output; generated JSON can be loaded again. Mark complete only after verifying the current files.

PHASE 2 | REALISTIC SYNTHETIC MATCH GENERATOR

Goal: Generate coherent, formation-aware 90-minute match sequences.

- Support 4-3-3, 4-2-3-1, 4-4-2 and 3-5-2 with role-aware player action selection.
- Connect events into possessions: successful passes/carries continue the sequence; failed actions, tackles and recoveries can transfer possession; shots and restarts have explicit semantics.
- Use one shared pitch coordinate convention for both teams. Change attacking direction at half-time; never label the same coordinates as team-relative in one component and global in another.
- Add controlled scenarios: high press, counterattack, defensive dominance and harmless possession.

Exit check: A seeded match has connected possession IDs, plausible event progression, legal coordinates, half-time direction changes and reproducible scenario settings.

PHASE 3 | DATA VALIDATION AND AUTOMATED TESTS

Goal: Reject malformed events before they reach analytics.

- Validate event references, timestamps, team/player ownership, coordinate bounds, endpoint rules and allowed event types.
- Check possession continuity and transitions, monotonic event ordering, period boundaries and scenario configuration.
- Create unit tests and small hand-authored fixtures for edge cases; verify identical output for identical seeds.

Exit check: Valid fixtures pass; deliberately invalid data fails with actionable messages; deterministic and spatial checks run automatically.

PHASE 4 | MOMENTUM ANALYTICS AND SHIFT DETECTION

Goal: Compute transparent team momentum from validated events.

- Calculate team state in a five-minute rolling window, updated every 30 seconds, with recency weighting.
- Initial 0-100 component weights: territory 20%, possession value/progression 25%, pressure/recoveries 20%, chance creation 25%, possession control 10%.
- Implement shift rules with persistence, supporting-event requirements and hysteresis/cooldown to avoid duplicate markers.

Exit check: Known event fixtures produce expected feature changes; shifts can be reproduced and traced to metric calculations. Tune weights and thresholds from evaluation.

PHASE 5 | PLAYER ATTRIBUTION AND TACTICAL EVIDENCE

Goal: Explain which players contributed to a detected shift and why.

- Rank primary and secondary contributors from measured event-level contributions, not from narrative guesses.
- Use provisional roles such as Momentum Catalyst, Progression Driver, Chance Creator, Defensive Trigger and Finisher.
- Reconstruct a chronological 5-7 event evidence chain and include before/after metrics in a structured fact pack.

Exit check: A known catalyst ranks highly in a designed test, evidence links to real event IDs, and attribution values are clearly described as model estimates.

PHASE 6 | MICROSOFT AI AND PERSONALIZED EXPLANATIONS

Goal: Turn verified facts into useful audience-specific explanations.

- Use Microsoft Agent Framework for agent/orchestration as needed and Microsoft Foundry for model access/runtime.
- Provide only validated fact packs; constrain outputs to the evidence and request citations/references to event IDs where possible.
- Offer casual, tactical and player-focused phrasing without changing underlying facts.

Exit check: Grounding tests show explanations do not fabricate details and all material claims map to the fact pack.

PHASE 7 | INTERACTIVE DASHBOARD AND MATCH EXPLORATION

Goal: Make the analysis explainable during a short demo.

- Show a match timeline with detected shift markers and a before/after summary.
- Allow a user to inspect contributors, roles, metric changes and the ordered evidence chain.
- Add pitch/player highlighting if it improves the minimum judging flow.

Exit check: A reviewer can follow Shift -> Attribution -> Evidence -> Explanation without needing the developer to interpret hidden output.

PHASE 8 | INTEGRATION, DEPLOYMENT AND DOCUMENTATION

Goal: Connect components and prepare a robust local-first demo.

- Integrate the validated generator, analytics, attribution, fact pack, UI and AI explanation path.
- Consider Azure deployment or Foundry Hosted Agents only if access, cost and time permit; keep local execution as fallback.
- Document configuration, installation, environment variables and reproducible run steps.

Exit check: The end-to-end path works from a clean setup, with secrets kept out of Git and a reliable local fallback.

PHASE 9 | FINAL VERIFICATION AND SUBMISSION

Goal: Prove the core behaviour and prepare the hackathon submission.

- Run tests for generation, validation, momentum, shift detection, attribution, evidence ordering and AI grounding.
- Demo all controlled scenarios and document expected vs actual outcomes.
- Update README and architecture docs, produce a concise demo video/script and verify official submission requirements.

Exit check: A fresh run reproduces the demo; core claims are demonstrated rather than merely listed.

Minimum demo gate

Do not advance to the AI/UI polish phase while core scenarios still fail. First prove seeded events -> validation -> momentum -> shift -> attribution -> evidence on known fixtures.

Status note: this roadmap defines planned acceptance criteria. A phase is complete only when its code exists, automated checks pass, and the relevant README/docs status is updated.